

Advanced Computer Systems Project 1 Report

Ivan Cheng (ECSE)

Rensselaer Polytechnic Institute

ECSE 4320 Advanced Computer Systems

Dr. Tong. Zhang

December 15th, 2025

Feature 1: SMT Interference

Overview

Simultaneous Multithreading (SMT), allows two hardware threads to share one physical core. While SMT improves utilization for latency-hidden workloads, it can also hurt performance when both hardware threads compete for shared execution resources.

The goal of this feature is to evaluate the effect of SMT on two different microbenchmarks:

- Compute-bound kernel – dominated by arithmetic operations
- Memory-bound kernel – dominated by streaming memory accesses

The central question is:

- Does SMT hurt or help performance for these two classes of workloads, and how does it compare to running threads on separate physical cores?

Experimental Method

I used *taskset* to pin threads to specific logical CPUs. On my CPU:

- CPU 0 and CPU 1 share the same physical core
- CPU 2 is a different physical core

Each benchmark was run under three scenarios:

1. Solo (core 0) – one thread on CPU 0
2. SMT same core (0,1) – two threads sharing a physical core
3. Diff cores (0,2) – two threads on separate physical cores

Parameters were held constant across all runs:

- Array size: 1,000,000 elements
- Iterations: 200
- Throughput metric: updates/second

Commands that were used in experiment:

- `taskset -c 0 ./smt_bench compute 1 1000000 200`
- `taskset -c 0,1 ./smt_bench compute 2 1000000 200`
- `taskset -c 0,2 ./smt_bench compute 2 1000000 200`
- `taskset -c 0 ./smt_bench memory 1 1000000 200`
- `taskset -c 0,1 ./smt_bench memory 2 1000000 200`
- `taskset -c 0,2 ./smt_bench memory 2 1000000 200`

Results

Table 1 — Compute Kernel Throughput

Scenario	Threads	Time (s)	Updates/s
Solo (core 0)	1	0.654585	3.055×10^8
SMT same core (0,1)	2	0.327849	6.100×10^8
Diff cores (0,2)	2	0.328682	6.084×10^8

Compute Kernel Throughput (Updates/sec)

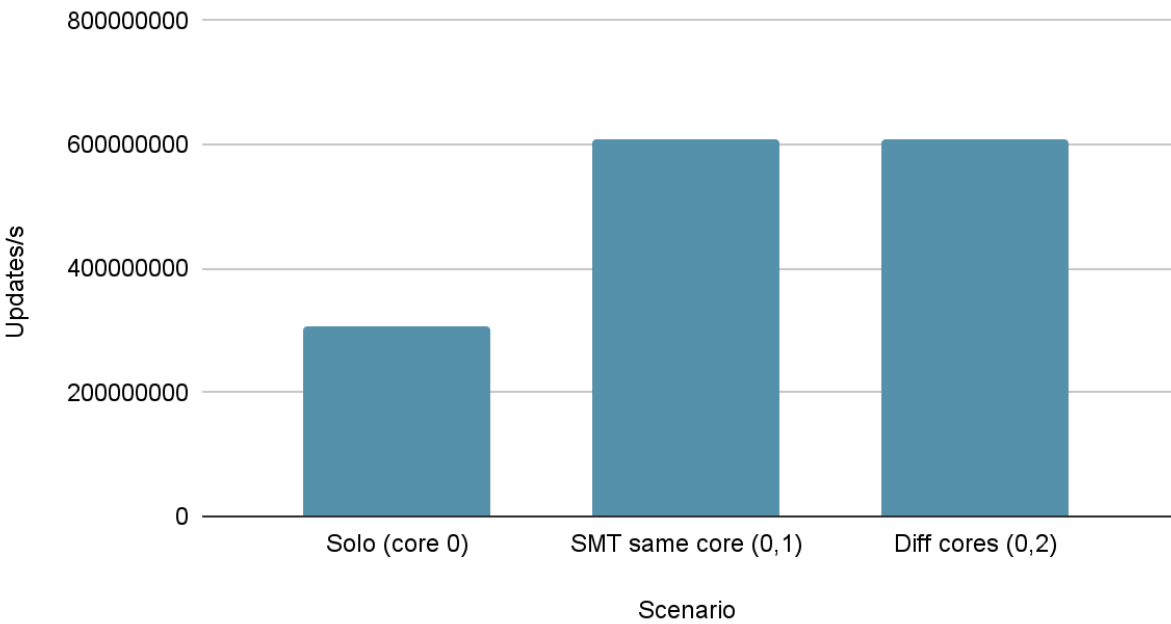
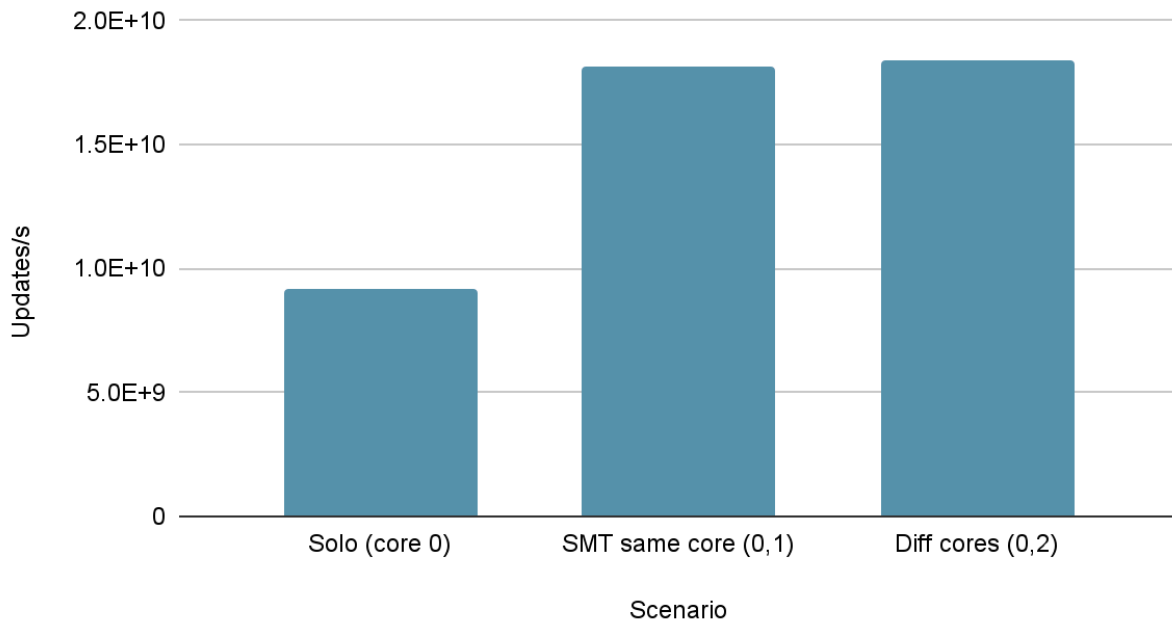


Table 2 — Memory Kernel Throughput

Scenario	Threads	Time (s)	Updates/s
Solo (core 0)	1	0.0218719	9.144×10^9
SMT same core (0,1)	2	0.0110222	1.815×10^{10}
Diff cores (0,2)	2	0.0108749	1.839×10^{10}

Memory Kernel Throughput (Updates/sec)



Analysis

Compute Kernel

The compute kernel shows almost perfect scaling from one to two threads, regardless of whether the threads share the same physical core or not. This indicates that the kernel does not saturate the core's execution units and that SMT interference is minimal because each hardware thread can still issue operations without problems arising. This occurs because the compute kernel does not fully saturate the core's execution units which allows both hardware threads to make forward progress without significant resource contention.

Memory Kernel

The memory kernel also scales nearly linearly. I observed that solo throughput is already extremely high. Both SMT and separate-core configurations nearly double throughput which implies the memory system is not saturated with just one thread.

Conclusion

For both compute-bound and memory-bound kernels, SMT interference was negligible. In all scenarios, adding a second thread doubled throughput. Key takeaway was that SMT does not always degrade performance. In workloads that don't overuse compute units, SMT can deliver almost ideal throughput scaling. A different workload configuration that is larger and has a higher intensity might show much stronger SMT contention.

Feature 2: CPU Affinity and Scheduler Behavior

Overview

CPU affinity controls how threads are assigned to physical cores. When threads are pinned to specific cores, they maintain cache locality and avoid migration overhead, while allowing the OS to schedule threads freely which can introduce variability.

The goal of this experiment is to quantify how thread placement affects throughput for a CPU-bound workload by comparing three scheduling configurations: all threads pinned to the same core, each thread pinned to a separate core, and the default Linux scheduler behavior.

The central question for this feature is how does thread placement affect the throughput of a CPU-bound workload?

Experimental Method

I used `affinity_bench` microbenchmark, which performs a large number of integer operations using multiple threads.

I evaluated three different CPU affinity configurations:

1. Same Core, all four threads pinned to logical CPU 0
2. Different Cores, each thread pinned to a separate logical CPU
3. Default Scheduler, threads allowed to run anywhere within CPUs 0–3

Parameters were held constant across all runs:

- 4 threads

- 50,000,000 iterations per thread, 200000000 iterations per thread
- 400 total million operations across threads

Commands that were used in experiment:

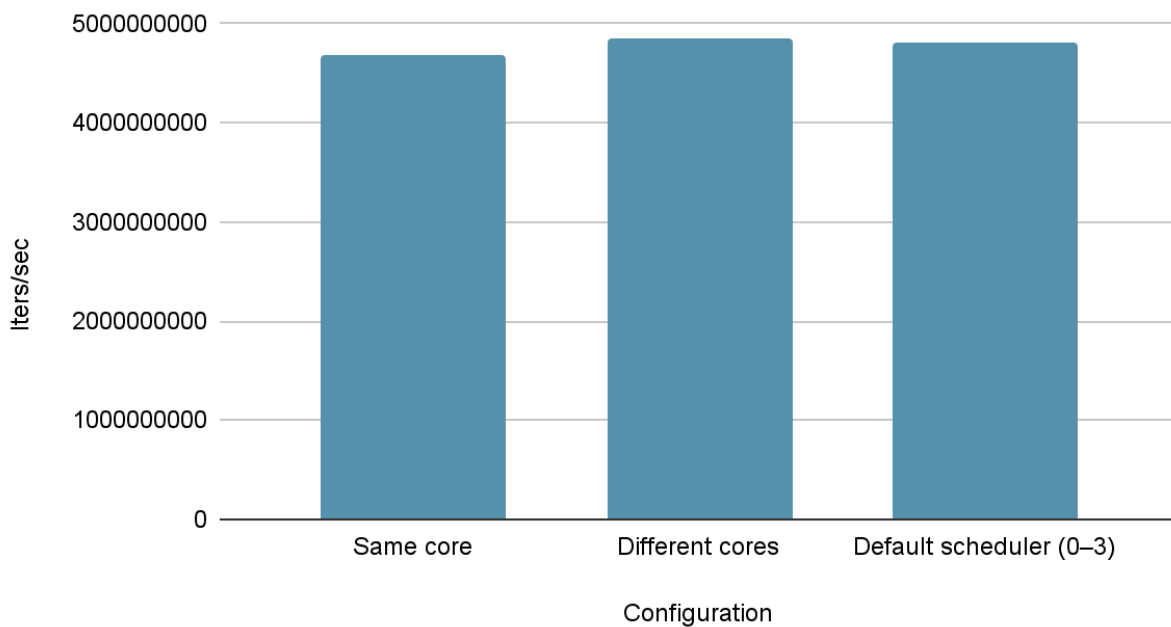
- `./affinity_bench 4 0 0 0 0 50000000`
- `./affinity_bench 4 0 1 2 3 50000000`
- `taskset -c 0-3 ./affinity_bench 4 0 1 2 3 50000000`
- `./affinity_bench 4 0 0 0 0 200000000`
- `./affinity_bench 4 0 1 2 3 200000000`
- `taskset -c 0-3 ./affinity_bench 4 0 1 2 3 200000000`

Results

Table: CPU Affinity Benchmark Results

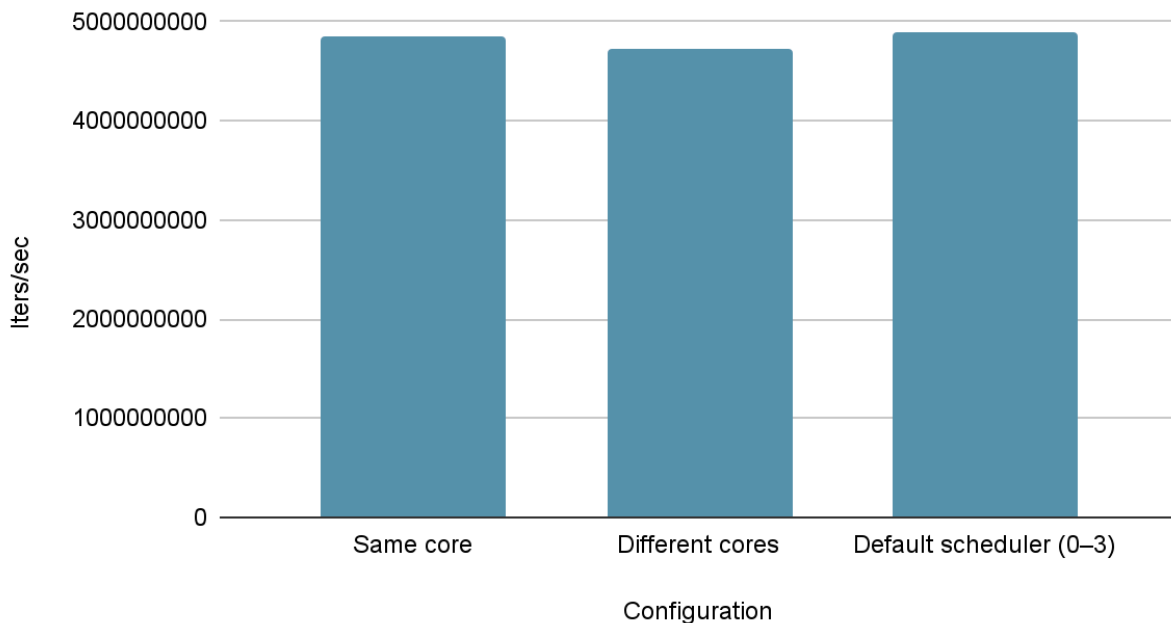
Configuration	Mapping	Time (s)	Iters/sec
Same core	0,0,0,0	0.0427626	4.677×10^9
Different cores	0,1,2,3	0.0412850	4.844×10^9
Default scheduler (0-3)	OS chooses	0.0415976	4.808×10^9

CPU Affinity Throughput Comparison (Iters/sec)



Configuration	Mapping	Time (s)	Iters/sec
Same core	0,0,0,0	0.164535	4.862×10^9
Different cores	0,1,2,3	0.169613	4.717×10^9
Default scheduler (0–3)	OS chooses	0.163181	4.903×10^9

CPU Affinity Throughput Comparison 200m Iterations (Iters/sec)



Analysis

Increasing the workload from 50M to 200M iterations per thread did not generally change the performance gap between affinity configurations.

Same-Core Configuration

When all four threads are pinned to CPU 0, they must share a single core's execution units, L1/L2 caches, and decode/issue pipelines.

Despite this, the throughput only dropped slightly to 4.677×10^9 iters/sec, demonstrating that this benchmark is relatively lightweight. The CPU is able to interleave threads efficiently without major stalls.

Different-Core Configuration

Pinning each thread to its own physical or logical core maximizes parallelism. This configuration achieved the highest throughput: 4.844×10^9 iters/sec. Distributing threads eliminates contention for private resources so this result represents near-ideal scaling.

Default Scheduler

Allowing the OS scheduler to decide where each thread runs resulted in 4.808×10^9 iters/sec, nearly identical to the explicitly pinned different-core configuration. This shows that Linux's scheduler does a good job of spreading threads across cores to maximize throughput.

Occasional thread migrations may explain why this configuration is slightly slower than perfect pinning. This indicates that the Linux scheduler distributes threads across available cores for this workload effectively which minimizes the need for manual affinity tuning.

Conclusion

These results show that CPU affinity has only a modest impact on this workload. All three scenarios achieved nearly identical throughput. This behavior indicates that the workload is compute-light and does not saturate the integer execution units, caches, or pipelines structures.

Feature 3: I/O Data Path

Overview

This feature investigates how different I/O data paths affect application-level performance when reading large files. Two approaches were compared: buffered I/O using the `read()` system call and memory-mapped I/O using `mmap()`.

The central goal of this experiment is to determine whether memory-mapped I/O delivers higher throughput than read-based I/O for large sequential file access in a virtualized environment.

The central question is how the choice of file I/O mechanism `read()` vs `mmap()` affects file-read throughput and execution time for large sequential workloads.

Experimental Method

A custom `io_bench` microbenchmark was used which performs large sequential file reads using two different I/O mechanisms.

Two different I/O data path configurations were evaluated:

1. Buffered I/O (read() system call) – the file is read into a user-space buffer in fixed-size blocks
2. Memory-Mapped I/O (mmap() system call) – the file is mapped into the process address space and accessed directly

Parameters were held constant across all runs:

- 1 GiB input file
- Sequential access pattern
- 64 KiB block size for read() mode
- Single thread execution

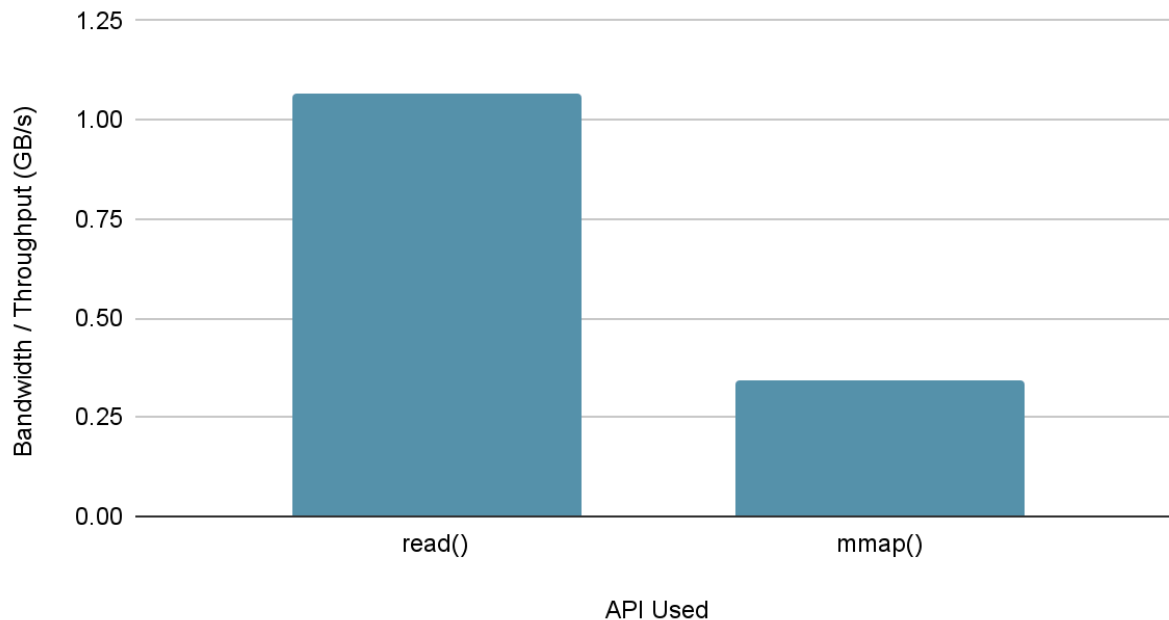
Commands that were used in the experiment:

- ./io_bench read testfile.bin 65536
- ./io_bench mmap testfile.bin

Results

Method	API Used	Block Size	Time (s)	Bandwidth (GB/s)
Buffered read	read()	64 KiB	1.006	1.067
Memory-mapped	mmap()	N/A	3.148	0.341

File Read Throughput: read() vs mmap()



Analysis

Contrary to the expectation that memory-mapped I/O should outperform buffered I/O, the `read()`-based implementation achieved significantly higher throughput. Buffered I/O reached approximately 1.07 GB/s, while the memory-mapped I/O achieved only 0.34 GB/s. This behavior may be due to the execution environment. The experiment was run under Windows subsystem for Linux, which implements Linux system calls through a virtualization layer on top of the Windows kernel. This results in `mmap()` to undergo high overhead due to frequent page faults and translation between Linux and Windows memory management. The `read()` path benefits from aggressive kernel buffering. As a result, buffered I/O achieved substantially higher effective throughput in this environment. This result is specific to the WSL environment where `mmap()` incurs additional virtualization and page fault overhead that would be less significant on native Linux systems.

Conclusion

This experiment demonstrates that memory-mapped I/O does not universally outperform buffered I/O. In a virtualized environment such as WSL, `read()`-based file access provides significantly higher throughput than `mmap()`. The findings emphasize that I/O performance depends not only on the API used by the application but also on how the operating system and platform implement that API.

Feature 4: Cache Prefetcher Effects

Overview

This feature investigates how CPU hardware cache prefetchers influence performance under different memory access patterns. Modern processors attempt to predict future memory accesses and fetch data into the cache ahead of time. This experiment evaluates how memory stride affects overall memory throughput and evaluates the impact of prefetcher behavior on system performance.

The central question for this feature is how does memory access stride impact cache prefetcher effectiveness and overall memory throughput on a modern CPU?

Experimental Method

A custom `stride_bench` microbenchmark was used which performs large memory scans using fixed-stride access patterns.

Multiple memory-access stride configurations were evaluated:

1. Small stride (cache-line-sized accesses)
2. Medium stride (multiple cache lines between accesses)
3. Large stride (page-sized gaps between accesses)

Parameters were held constant across all runs:

- Fixed array size allocated in memory
- Single-threaded execution
- Read-only memory access pattern
- Constant iteration count

Commands that were used in the experiment:

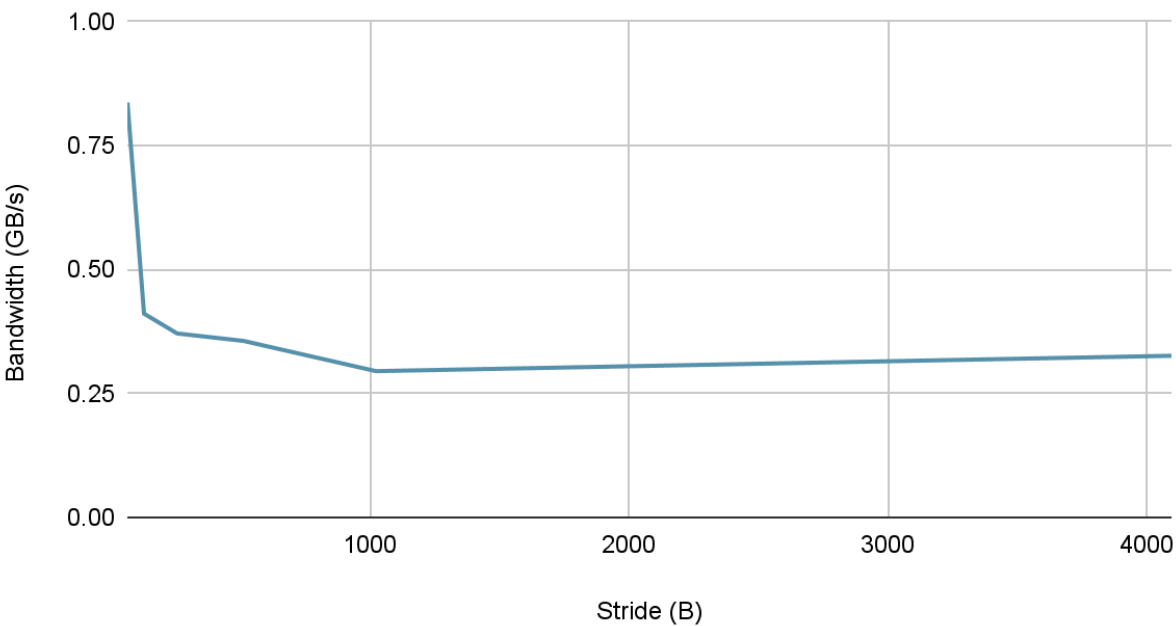
- `./stride_bench 256 64 20 2`
- `./stride_bench 256 128 20 2`
- `./stride_bench 256 256 20 2`
- `./stride_bench 256 512 20 2`
- `./stride_bench 256 1024 20 2`
- `./stride_bench 256 4096 20 2`

Results

Stride (B)	Bandwidth (GB/s)
------------	------------------

64	0.837
128	0.411
256	0.371
512	0.356
1024	0.295
4096	0.326

Cache Prefetcher Effectiveness vs Memory Stride



Analysis

The results show that memory access stride has a strong impact on effective bandwidth. At 64 bytes, throughput was highest at approximately 0.84 GB/s, indicating that the hardware prefetcher was effective at detecting the predictable access pattern. When the stride increased to 128 bytes, throughput dropped to 0.41 GB/s, indicating reduced prefetch accuracy. As stride increased to 256 and 512 bytes, throughput decreased. At these stride lengths, memory accesses span multiple cache lines and reduce spatial locality, limiting the effectiveness of hardware prefetching. These results demonstrate the strong dependence of modern CPU performance on predictable memory access patterns and effective cache prefetching.

Conclusion

The stride benchmark demonstrates that hardware cache prefetchers are effective for small, predictable memory access strides but become less effective as stride size increases. Larger strides reduce spatial locality and prevent the processor from accurately prefetching future cache lines, leading to lower memory throughput.

References

- [1]OpenAI, "ChatGPT," chatgpt.com, <https://chatgpt.com>
- [2]Wikipedia Contributors, "Simultaneous multithreading," Wikipedia, Jan. 27, 2020.
https://en.wikipedia.org/wiki/Simultaneous_multithreading
- [3]Wikipedia Contributors, "Processor affinity," Wikipedia, Nov. 17, 2025.
- [4]H. Newman, "A Trip Down the Data Path: I/O and Performance | Enterprise Storage Forum," Enterprise Storage Forum, Jan. 15, 2003.
<https://www.enterprisestorageforum.com/hardware/a-trip-down-the-data-path-i-o-and-performance/>
- [5]H. Newman, "A Trip Down the Data Path: I/O and Performance | Enterprise Storage Forum," *Enterprise Storage Forum*, Jan. 15, 2003.
<https://www.enterprisestorageforum.com/hardware/a-trip-down-the-data-path-i-o-and-performance/>